

GEXLens — Manuál pro správce a vývojáře

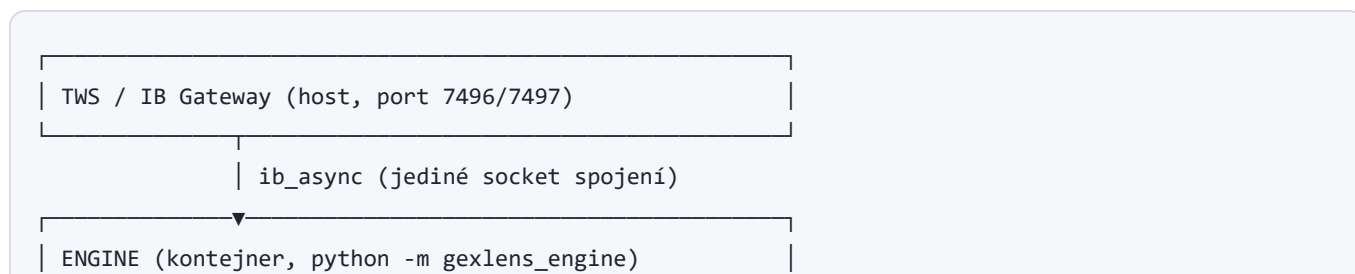
Verze 1.1 · srpen 2026 · interní dokumentace — není dostupná v aplikaci

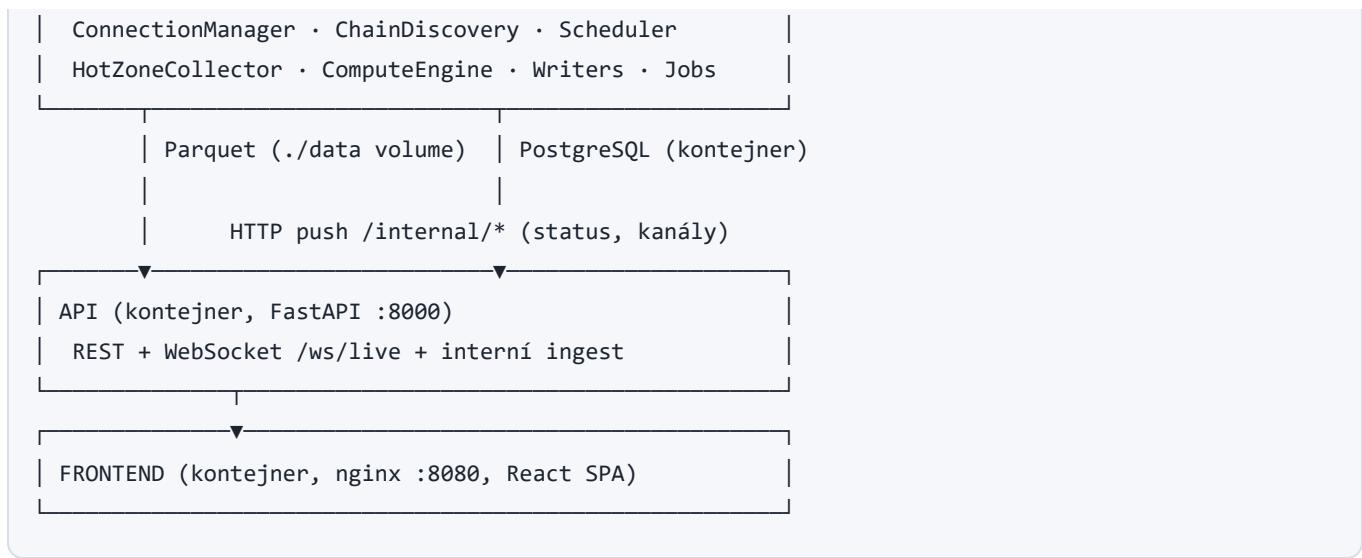
Technický popis architektury, provozu, konfigurace a vývoje aplikace GEXLens. Uživatelská příručka: `UZIVATELSKY-MANUAL.md`. Zdroj pravdy funkčních požadavků: [docs/SPEC.md](#) (v2.0); architektonická rozhodnutí v [docs/adr/](#).

Obsah

1. [Architektura](#)
 2. [Struktura repozitáře](#)
 3. [Provoz \(docker compose\)](#)
 4. [Konfigurace — kompletní reference](#)
 5. [Engine — datová pipeline](#)
 6. [Datové formáty a persistence](#)
 7. [API reference](#)
 8. [Frontend](#)
 9. [Vývojové prostředí](#)
 10. [Testy a CI](#)
 11. [Známé limity účtu a otevřené body](#)
 12. [Diagnostics a údržba](#)
 13. [Zprovoznění od nuly — IBKR účet, TWS/Gateway](#)
 14. [Bezpečnost a nasazení na server](#)
-

1. Architektura





Klíčové vlastnosti:

- **Engine a API jsou oddělené procesy.** Engine počítá a zapisuje; API jen čte storage a přeposílá live push z enginu (interní HTTP ingest → StatusStore + LiveHub → WebSocket klientům).
- **Vše lokální** — API CORS povoluje jen `localhost / 127.0.0.1` ; žádná telemetrie.
- Engine se z kontejneru připojuje na TWS na hostiteli přes `host.docker.internal` .

2. Struktura repozitáře

GEX/	
└ engine/	Python 3.12 balík gexlens_engine
├ └ src/gexlens_engine/	
├ └ config.py	Pydantic Settings (.env, GEXLENS_*)
├ └ ibkr/	connection, discovery, scheduler, hotzone,
├ └ └	underlying (bary+pacing), mock (pro testy)
├ └ compute/	gex, levels, heatmap, walls, cumdelta, profile
├ └ storage/	parquet_store, oi_archive, retention, meta
├ └ adapters.py	produkční ib_async adaptéry + HttpPublisher
├ └ runtime.py	EngineRuntime – minutový cyklus (testovatelný)
├ └ └ __main__.py	vstupní bod: discovery→archiv→smyčka
└ api/	Python balík gexlens_api (FastAPI)
├ └ src/gexlens_api/	main (routy+WS), data, heatmap (vektorizace),
├ └ └	live (hub), status, crud, alerts, meta_repo
└ frontend/	React + TypeScript + Vite
├ └ src/	components/, heatmap/, replay/, panels/,
├ └ └	profile/, annotations/, state/, api/
└ docs/	SPEC.md, adr/, manual/
└ docker/	Dockerfiles + nginx.conf
└ compose.yml	celý stack

```
|─ scripts/                bootstrap, start skript pro plochu
└─ Makefile                test / run / run-api / run-frontend / run-engine
```

Pravidla vývoje jsou v [CLAUDE.md](#) : práce po GitHub issues, golden testy výpočtů, IBKR se v CI nikdy nevolá živě (mock vrstva `engine/ibkr/mock.py`), komentáře česky / identifikátory anglicky.

3. Provoz (docker compose)

```
docker compose up -d --build      # start / rebuild
docker compose ps                 # stav služeb
docker compose logs -f engine    # živé logy enginu
docker compose stop               # zastavení (data zůstávají)
docker compose down              # odstranění kontejnerů (volume pgdata zůstává)
```

Služba	Port (host)	Poznámka
frontend	8080	nginx, SPA + <code>/manual/</code> wiki
api	8000	FastAPI, OpenAPI na <code>/docs</code>
postgres	55432	⚠ záměrně ne 5432/5433 — na vývojovém PC běží nativní PostgreSQL na obou
engine	—	bez portu; TWS přes <code>host.docker.internal:7496</code>

Data: Parquet v `./data` (bind mount, sdílené engine↔API), PostgreSQL ve volume `pgdata` .
Zálohovat stačí `./data` + `pg_dump` (hlavně tabulku `oi_eod` , která se nikdy nemaže).

Provozní detaily kontejnerů

- Kontejnery běží pod **UID 10001** (`docker/entrypoint.sh`) — bind-mount adresáře musí být zapisovatelné pro tento UID.
- **nginx frontendu proxuje** `/api` na službu API — port API se ven nepublikuje; po nasazení frontendu je potřeba **hard reload** (Ctrl+Shift+R), nginx drží starý bundle.
- Shellové skripty mají v `.gitattributes` vynucené `eo1=lf` — checkout na Windows je nesmí konvertovat na CRLF (kontejner by je nespustil.)

4. Konfigurace — kompletní reference

Zdroj: proměnné prostředí `GEXLENS_*` a `.env` (viz `.env.example`). Validuje se při startu — nevalidní hodnota = engine odmítne nastartovat se srozumitelnou chybou.

Proměnná	Default	Význam
<code>GEXLENS_IBKR_HOST</code>	127.0.0.1	V compose přepsáno na <code>host.docker.internal</code>
<code>GEXLENS_IBKR_PORT</code>	7496	7496 live / 7497 paper (TWS); 4001/4002 (Gateway)
<code>GEXLENS_IBKR_CLIENT_ID</code>	1	
<code>GEXLENS_MARKET_DATA_TYPE</code>	1	1=live; delayed engine odmítá
<code>GEXLENS_CONNECT_TIMEOUT_S</code>	10	
<code>GEXLENS_RECONNECT_BACKOFF_BASE_S</code> / <code>_MAX_S</code>	2 / 60	Exponenciální reconnect
<code>GEXLENS_HEARTBEAT_INTERVAL_S</code> / <code>_TIMEOUT_S</code>	30 / 15	Heartbeat spojení; agresivnější hodnoty vedly k falešným reconnectům během sweep dávek
<code>GEXLENS_SYMBOLS</code>	ES	Základní sada futures podkladů (čárkami); watchlist z DB se přidává za běhu (ADR-0003)
<code>GEXLENS_MAX_INSTRUMENTS</code>	3	Strop souběžných instrumentů (rozpočet market data lines)
<code>GEXLENS_WATCHLIST_POLL_CYCLES</code>	5	Watchlist + runtime nastavení (<code>strike_range_points</code>) se čtou z DB každý k-tý cyklus
<code>GEXLENS_OI_ARCHIVE_EXPIRIES</code>	5	Ranní OI archiv pokrývá N nejbližších expirací (základ Δ OI vs. včera)

Proměnná	Default	Význam
GEXLENS_SWEEP_NEXT_EXPIRY	true	Sekundární sweep následující expirace (positioning příští seance)
GEXLENS_NEXT_EXPIRY_SWEEP_EVERY	3	Kadence sekundárního sweepu (každá k-tá minuta)
GEXLENS_STRIKE_RANGE_POINTS	200	Výchozí denní obálka spot $\pm$ X (ADR-0002)
GEXLENS_STRIKE_RANGE_EXPAND_THRESHOLD	0.25	Rozšíření při přiblížení k okraji
GEXLENS_STRIKE_RANGE_MAX_POINTS	800	Strop šířky obálky ($\geq 2 \times$ base)
GEXLENS_BATCH_SIZE	80	Dávka rotačních subskripcí
GEXLENS_BATCH_TIMEOUT_S	4	Čekání na kompletní data kontraktu
GEXLENS_WINGS_SWEEP_EVERY	3	Křídla každý k-tý cyklus
GEXLENS_ATM_SWEEP_WIDTH	30	ATM $\pm$ N strikes každý cyklus
GEXLENS_REPAIR_MAX_ATTEMPTS	3	Retry repair fronty za sweep
GEXLENS_MARKET_DATA_LINES	100	Kapacita market data lines — tvrdý strop účtu je 100 (změřeno #609; původní odhad „ ≥ 150 “ z ADR-0001 neplatil). <code>batch_size</code> nikdy nezvyšovat
GEXLENS_HOT_ZONE_WIDTH	15	Cílová šířka hot zóny (degraduje dle účtu)
GEXLENS_TICK_BY_TICK_MAX_STREAMS	5	Naměřený limit účtu (ADR-0001)
GEXLENS_DATABASE_URL	postgres localhost	V compose směřuje na službu <code>postgres</code>
GEXLENS_DATA_DIR	data	Kořen Parquet partic

Proměnná	Default	Význam
GEXLENS_RETENTION_DAYS	90	Purge okno (ADR-0022, odchylka od R3). Nemaže se: oi_eod ani žádná bars/partice
GEXLENS_KEEP_BARS_FOREVER	true	Bary podkladu se z purge vyjímají — základ historických výpočtů (ADR-0028)
GEXLENS_DISK_LIMIT_GB	2	Alert při překročení
GEXLENS_RETENTION_PURGE_TIME_UTC	21:30	Čas nočního purge
GEXLENS_API_BASE	http://127.0.0.1:8000	Kam engine pushuje (v compose http://api:8000)

| GEXLENS_PG_PASSWORD | — | **Povinné** — compose bez něj nenastartuje (generuje scripts/init-secrets.ps1) | | GEXLENS_API_TOKEN | — | **Povinné** — sdílené tajemství /internal/* a /backup/postgres | | GEXLENS_BIND_ADDR | 127.0.0.1 | Na jaké adrese publikují porty (server: ponechat loopback + reverse proxy) | | GEXLENS_ALLOWED_ORIGINS | — | CORS whitelist API | | GEXLENS_NEWS_API_TOKEN | — | Token news-engine → API push | | GEXLENS_TASTY_SHADOW | false | Shadow porovnání tastytrade feedu (M7 fáze 1, #613) — zapisuje JEN do feed_comparison, nic nepublikuje. Kill switch = vypnout flag | | GEXLENS_TASTY_CLIENT_SECRET / _REFRESH_TOKEN | — | OAuth2 grant **výhradně scope read** (ADR-0025); dev grant patří do .env.dev pod standardními názvy. Obsah .env se nikdy nevypisuje do konzole | | GEXLENS_CROSSCHECK_ENABLED | true | Křížová kontrola IBKR × tasty (#517 fáze A) — pasivní, bez requestů a linek navíc. Bez shadow se tiše nezapne | | GEXLENS_CROSSCHECK_SHARE_THRESHOLD / _MINUTES / _COOLDOWN_MINUTES | 0.70 / 3 / 15 | Prahy **měřené** na shadow historii, ne odhadnuté — viz níže. _MINUTES pod 3 = falešný poplach každou třetí minutu | | GEXLENS_TASTY_SPOT_FALLBACK | true | Cena podkladu z tastytrade, když IBKR přestane posílat ticky (#614 fáze 2a) | | GEXLENS_TASTY_SPOT_STALE_AFTER_S / _RECOVER_AFTER_S / _MAX_AGE_S | 30 / 60 / 30 | Hystereze spotu: kdy převzít, kdy vrátit, max stáří tasty kotace. Návrat je delší než převzetí schválně — kmitání zdroje stojí víc než o půl minuty pozdější návrat | | GEXLENS_TASTY_CHAIN_FALLBACK | true | Fallback **celého opčního řetězu** (#614 fáze 2b). Spouští ho verdikt křížové kontroly, takže dědí její měřené prahy | | GEXLENS_TASTY_CHAIN_RECOVER_MINUTES | 5 | Kolik čistých minut v řadě vrátí řetěz na IBKR. Delší než zapínací série (_CROSSCHECK_MINUTES): přepnutí překreslí celý profil | | GEXLENS_TASTY_CHAIN_MAX_AGE_S | 120 | Max stáří tasty hodnoty, aby kontrakt vstoupil do fallbackového řetězu | | GEXLENS_STARTUP_CONNECT_WAIT_S | 60 | Jak dlouho se při startu čeká na IBKR, než engine rozjede zbytek i bez něj (#756). **Není to timeout spojení** — supervisor se pokouší dál. 0 = nečekat | | GEXLENS_TASTY_OI_FILL | true | Díry denního OI archivu

doplňí tasy `Summary` (#664) — typicky 0DTE ráno, než CME publikuje | | `GEXLENS_NEWS_LLM_ENABLED` | `false` | LLM klasifikace zpráv (Gemini). **Zakonzervováno** (#740 fáze 0): v `news_weights` neprošla Wilson gate ani jednou (0/20 řádků, hit rate 0,484 proti 0,516 u pravidel) | | `GEXLENS_NEWS_ALPACA_KEY_ID` / `_SECRET` | — | Alpaca News API — živý stream i historický backfill (#743, #744). Stačí paper účet a **Trading API**, ne Broker API |

Od #696 jde do kontejnerů celý `.env` (`env_file`), ne ruční výčet — každý klíč z `.env.example` po `docker compose up -d <služba>` skutečně platí. `environment:` v compose nese jen odvozené hodnoty (`DATABASE_URL`, adresy služeb, `/app/data`) a ty mají přednost. **Dev stack navíc čte volitelný** `.env.dev` (přepisy jen pro dev: vlastní tasy grant, symboly...; viz `.env.dev.example`). Pozn.: změna PG hesla v `.env.dev` vyžaduje reseed dev volume.

Frontend build-time: `VITE_API_BASE` (nginx build arg, default `http://127.0.0.1:8000`).

5. Engine — datová pipeline

Minutový cyklus (`runtime.EngineRuntime.run_cycle`):

1. **Sweep** — `SubscriptionScheduler` projede řetězec v dávkách ($ATM \pm 30$ každý cyklus, křídla každý 3.), nekompletní kontrakty přes repair frontu, výsledek do in-memory cache.
2. **Snapshot** — cache → `SnapshotRow` (OI z ranního archivu) → atomický zápis Parquet.
3. **Výpočty** — GEX per strike (naivní dealer model, vyměnitelná strategie) → levels (flip interpolovaně, walls, centroid) → zápis do `derived/levels`.
4. **Cum Δ** — bar větev ($\Delta Vol \times \text{midpoint test} \times \Delta \times M$); hot zóna tick-by-tick přispívá průběžně. `close_minute` → `derived/flow`.
5. **Bary podkladu** — 5s reqRealTimeBars agregované na 1min → `derived/bars`.
6. **Push do API** — `/internal/status` + kanály `levels.*`, `flow.*`, `price.*`.

Multi-instrument orchestrátor (ADR-0003)

`__main__` řídí **pipeline per podklad** (`instruments.InstrumentPipeline`): cílová sada = `GEXLENS_SYMBOLS` U watchlist z DB — změny chodí okamžitě přes PostgreSQL `LISTEN/NOTIFY` (kanál `gexlens_watchlist`, #207: API po zápisu notifikuje, orchestrátor se probudí ze sleep a nový symbol startuje do sekund; svíčky dne doplní backfill z #221), poll à `WATCHLIST_POLL_CYCLES` zůstává jako fallback pro backendy bez NOTIFY. Probuzení uprostřed minuty spustí plný cyklus jen pro nové pipeline — běžící by duplikovaly zápisy. Sweeepy instrumentů běží **sekvenčně** — špička market data lines je vždy jedna dávka. Multiplikátor a burza se čtou z contract details. Ne-futures symbol → alert `instrument_error` + cooldown 30 cyklů. Pád cyklu jednoho instrumentu neshodí ostatní; status se agreguje (součty Greeks/repair, pole `symbols`).

Každá pipeline navíc drží **sekundární runtime následující expirace** (`secondary=True`): sweep v kadenci `NEXT_EXPIRY_SWEEP_EVERY` , zapisuje jen snapshots + levels své expirace (flow/bary patří výhradně aktivnímu řetězu — soubory jsou per symbol).

Další joby: **OI archiv** při startu + retry à 30 min dokud den nemá data (alert `oi_missing`); pokrývá `OI_ARCHIVE_EXPIRIES` nejbližších expirací — základ Δ OI vs. včera. **POZOR: OI se čte přes generic tick 101 i pro FOP** (tick 588 na FOP nedodává nikdy — ADR-0001 v3; hodnota se čte podle strany kontraktu, opačná strana je validní 0.0). **Auto-rozšíření obálky strikes** (grow-only, capped → alert) + runtime změna `strike_range_points` ze Settings UI (překlopí pipeline). **Denní roll expirace**: vypršela pipeline se zastaví a další cyklus založí novou s čerstvou discovery (bezobslužný přechod přes víkend). **Noční retention purge** po `RETENTION_PURGE_TIME_UTC` .

Bary podkladu (#221): **Backfill 1min barů** při startu pipeline (aktuální den + retention okno, reqHistoricalData pod pacing guardem, upsert podle ts_min — živý stream a backfill se nedublují). **Hlídní tiché ztráty barů** (`BarsStallDetector`): když $\geq$ `BARS_STALL_ALERT_MINUTES` (default 3) nedorazí žádný 5s bar, ale spot se hýbe, odejde alert `bars_stalled` (typicky mrtvé TWS farmy po noční přestávce — pomáhá restart TWS); po návratu streamu alert `bars_recovered` + automatický re-backfill dnešního dne doplní díru. Bez pohybu spotu (zavřený trh) se nehlásí nic.

Odolnost: ConnectionManager watchdog (heartbeat 30/15 s + exponenciální reconnect + plná resubskripce — **vč. spot tickeru a realtime barů podkladu** přes `on_resubscribe`), spot fallback last → marketPrice → close (start i o víkendu), discovery s timeoutem a retry (sec-def farm výpadky), výjimka v cyklu nikdy neshodí smyčku, pacing guard historical requestů ($\leq 60/10$ min, dedup, prioritizace).

6. Datové formáty a persistence

Parquet (`GEXLENS_DATA_DIR` , **retence 90 dní — ADR-0022**)

Partice	Schéma
<code>snapshots/{sym}/{expiry}/{YYYY-MM-DD}.parquet</code>	ts_min, strike, right, bid, ask, last, volume, iv, delta, gamma, theta, vega, oi, stale_age
<code>ticks/{sym}/{YYYY-MM-DD}.parquet</code>	ts, conld, price, size, side
<code>derived/{sym}/{expiry}/levels/{date}.parquet</code>	ts_min, flip, call_wall, put_wall, centroid, total_gex

Partice	Schéma
<code>derived/{sym}/flow/{date}.parquet</code>	ts_min, flow_delta, cum_delta
<code>derived/{sym}/bars/{date}.parquet</code>	ts_min, open, high, low, close, volume — z purge vyňaté, drží se navždy (<code>GEXLENS_KEEP_BARS_FOREVER=true</code>); ES i NQ mají ~2 roky historie od 2024-07-28 (backfill <code>scripts/backfill_bars.py</code>)
<code>derived/{sym}/netflow/{date}.parquet</code>	Δ -vážený tok per strana (podklad FA odhadu OI)
<code>derived/{sym}/{expiry}/oiest/{date}.parquet</code>	FA odhad OI ($\text{netflow} \times \alpha$, #232)
<code>derived/{sym}/{expiry}/gexprofile(fa)/... +</code> <code>gexfield(fa)/...</code>	Dyn profily/pole; <code>...fa</code> varianty nad FA odhadem
<code>derived/{sym}/{expiry}/charmprofile/...</code> , <code>vannaprofile/...</code> (+ <code>...field</code>)	Dyn Charm/Vanna plochy (#204)
<code>derived/{sym}/{expiry}/greekssource/{date}.parquet</code>	Zdroj greeks per minutu (model/computed, #547)
<code>derived/{sym}/{expiry}/oimissing/{date}.parquet</code>	Striky bez OI (šrafura, #465)
<code>derived/{sym}/catchup/{date}.parquet</code>	Příznak dohánění po startu (#518)
<code>derived/{sym}/gexforward/{date}.parquet</code>	Forward GEX (#519): bloky per budoucí obchodní den (day, grid, values, dropped_expiries, dropped_share, iv_fallback_share); jen poslední stav, přepočít po OI archivu
<code>derived/sentiment/{SYM}/{date}.parquet</code>	1min řada SentIndexu per symbol (ADR- 0026; ploché soubory bez symbolu = ES legacy)

Zápis je **atomický** (temp + rename) — po pádu procesu nikdy nezůstane částečný soubor; osiřelé `.tmp` se uklízí při dalším zápisu. Writer po restartu navazuje na rozepsaný den.

PostgreSQL

Tabulka	Účel
<code>oi_eod(symbol, expiry, strike, right, date, oi, iv, delta, gamma, theta, vega, close_prem, und_price)</code>	Věčný denní snímek řetězce — od #519 nese vedle OI i IV/greeks/závěrečnou prémii/ref. spot z ranního průchodu (NULL = model nedodal). Žádná retence, žádné delete API
<code>gamma_cliff</code>	Denní odpad gammy po expiraci + metriky následující seance (#576, fáze měření)
<code>feed_comparison</code>	Shadow porovnání IBKR × tastytrade per (minuta, kontrakt, pole) — jen po dobu sběru M7 fáze 1 (#613)
<code>sentiment_daily</code> , <code>sentiment_waves</code> , <code>news_*</code> , <code>signals</code> , <code>signal_outcomes</code> , <code>track_record</code>	SentimentLens (per symbol od ADR-0026)
<code>setups</code>	Setupy vč. <code>context</code> JSON (od #575 nese <code>band_sharpness</code> / <code>band_sharpness_pct</code> / <code>band_depth</code>) a <code>mechanics_version</code>
<code>watchlist</code> , <code>alerts</code> , <code>annotations</code> , <code>settings</code>	CRUD přes API

7. API reference

Interaktivní dokumentace: <http://127.0.0.1:8000/docs> (OpenAPI).

REST

Endpoint	Popis
<code>GET /health</code> , <code>GET /status</code>	Liveness; agregovaný stav pipeline (<code>lines_utilization</code> je od #630 měřená špička). Pole <code>chain_source</code> / <code>spot_source</code> nesou aktivní zdroj dat (#614), <code>feed_crosscheck*</code> verdikt křížové kontroly (#517 A). Chybějící klíč znamená „neměří se“ , ne „je to v

Endpoint	Popis
	pořádku" — od #756 chodí status i bez jediné pipeline a <code>connection</code> nese skutečný stav spojení
GET <code>/gexforward/{symbol}</code>	Forward GEX bloky per budoucí den (#519)
GET <code>/bars/{symbol}?date=</code>	Lehké 1min OHLCV bary seance (#674/#678) — bez /replay balíku
GET <code>/oidelta/{symbol}/{expiry}</code>	Δ OI posledních dvou archivovaných dnů + top movers (#674)
GET <code>/journal</code> , POST/PATCH/DELETE <code>/journal/*</code>	Deník tradera (#673, fáze A)
GET <code>/gammacliff/{symbol}</code>	Dnešní odpad gammy + historie útesů (#576)
GET <code>/fa/alpha</code>	Kalibrovaná α FA odhadu per symbol (#232)
GET <code>/gexplane/{...}</code>	Dyn Charm/Vanna plochy (#204)
GET <code>/sentiment/*?symbol=</code>	Sentiment per symbol (ADR-0026): index/daily/state/waves
GET <code>/instruments</code> , GET <code>/instruments/{sym}/expiries</code>	Dostupné symboly/expirace (ze storage)
GET <code>/instruments/{sym}/days</code>	Uložené dny s expirací per den (Daily pohled)
GET <code>/profile/{sym}/aggregate?date</code>	Σ profil: OI/volume sečtené přes všechny expirace dne per strike (registrováno PŘED <code>/profile/{sym}/{expiry}</code>)
GET <code>/snapshots/{sym}/{expiry}?date&mode&scale&norm&from&to&raw</code>	Heatmap matice jako Arrow IPC stream ; <code>raw=true</code> = surová partice
GET <code>/levels/{sym}/{expiry}?date</code>	Časová řada flip/walls/centroid
GET <code>/profile/{sym}/{expiry}?date&ts&variant&oi_weight&spot</code>	Strike profil k okamžiku
GET <code>/flow/{sym}?date</code>	Cum Δ + OptVol + Vol řady
GET <code>/replay/{sym}/{expiry}/{date}</code>	Kompletní denní balík (levels/flow/bars JSON + snapshoty base64 Arrow + <code>oi_prev</code> pro Δ OI vs. včera)
CRUD <code>/watchlist</code> , <code>/alerts</code> , <code>/annotations?symbol&date</code> ,	PostgreSQL persistence

Endpoint	Popis
<code>/settings</code>	
<code>POST /internal/status</code> , <code>POST /internal/publish</code>	Ingest z engineu — vyžaduje hlavičku <code>X-GEXLens-Token</code> (#542)
<code>GET /backup/postgres</code>	Stream <code>pg_dump -Fc</code> — vyžaduje <code>X-GEXLens-Token</code> (#542)

WebSocket `/ws/live`

Protokol: klient pošle `{"action": "subscribe", "channels": ["status", "price.ES", "levels.*"]}` (podpora trailing wildcard), server vrátí ack a pushuje `{"channel": "...", "data": ...}`. Backpressure: fronta 100 zpráv per klient, při zaplnění se zahazují nejstarší framy. Kanály: `status`, `price.{sym}`, `snapshot.{sym}.{expiry}`, `levels.*`, `flow.*`, `alerts`, `news`.

Handshake kontroluje hlavičku `origin` (#542): CORS se na WebSocket nevztahuje, takže bez téhle kontroly by živý positioning četla libovolná stránka otevřená v prohlížeči. Povolen je same-origin (Host stránky za nginx), localhost v libovolném portu a cokoli v `GEXLENS_ALLOWED_ORIGINS`. Klienti bez hlavičky `origin` (engine, curl) projdou. Stropy: 64 souběžných spojení, 256 kanálů na spojení.

8. Frontend

- **Heatmapa:** data → offscreen bitmapa (překreslení jen při změně dat/módu), pan/zoom = GPU `drawImage` → 60 fps nezávisle na objemu; overlay canvas kreslí vektory (cena/svíčky, levels, walls, sessions, crosshair, anotace).
- **Replay:** `/replay` se stáhne jednou, `apache-arrow` dekoduje snapshoty, celý den se předpočítá v paměti (vč. profilu per minuta) — přetáčení je čisté krájení typed arrays. Timestampy se normalizují (`canonicalTs` — Arrow epoch vs. JSON ISO).
- **Stav:** React kontexty `AppState` (status z WS + REST initial fetch, view, téma, alerty) a `Crosshair` (sdílený všemi panely).
- **OI fallback:** při nulovém OI staví heatmapu z volume (engine mezitím posílá alert `oi_missing`).
- Wiki/manuál: statické HTML v `frontend/public/manual/` (generované z MD, viz níže) — servíruje ho vite dev i nginx.

9. Vývojové prostředí

Prerekvizity: [uv](#) (stáhne Python 3.12 sám), Node.js ≥ 20, Docker (pro PG integrační test lokálně volitelně).

```
uv sync --all-packages                # Python workspace (engine + api)
uv run ruff check .; uv run ruff format .  # lint/format
uv run mypy engine/src engine/tests api/src api/tests
uv run pytest                          # PG integrační test se přeskočí bez GEXLENS_TEST_PG_DSN

cd frontend; npm ci; npm run lint; npm test; npm run build
```

Dev servery: `make run-api` (uvicorn :8000), `make run-frontend` (vite :5173), `make run-engine` (vyžaduje TWS). CORS povoluje i :5173.

Regenerace manuálů (MD → HTML pro in-app wiki → PDF): `powershell scripts/build-manual.ps1` (vyžaduje Edge; PDF vzniká headless tiskem).

Konvence: feature branch `feat/{issue}-slug` / `fix/...`, PR s `closes #N`, merge po zeleném CI. Výpočty vždy s golden testy v `engine/tests/golden/` (ručně spočtené hodnoty, výpočet dokumentovaný v `description`).

Oddělená prostředí DEV a PROD (#568)

Vedle produkčního stacku (`compose.yml`, :8080) existuje dev stack (`compose.dev.yml`, projekt `gexdev`, :8081) s vlastním PG volume (`gexdev_pgdata`) a vlastní kopií parquet dat (`data-dev/`). Cíl: vývoj se nikdy nedotkne produkčních dat, která nejdou znovu pořídít (věčný OI archiv, setupy, track record).

Skript / ikona	Co dělá	Souběh s prod
<code>scripts/start-prod.ps1</code> (ikona GEXLens)	produkce; shodí dev-live, pokud běží	—
<code>scripts/start-dev.ps1</code> (ikona GEXLens DEV)	dev bez enginu: PG + API + frontend nad kopií dat	povolen — market data účtu se nedotkne, prod dál sbírá
<code>scripts/start-dev.ps1 - Live</code> (ikona GEXLens DEV+Engine)	plný stack proti TWS	zakázán — skript nejdřív shodí produkci (jeden účet); po dobu běhu prod nesbírá

Skript / ikona	Co dělá	Souběh s prod
<code>scripts/start-dev.ps1 -LiveTasty (start-gexlens-dev-live-tasty.cmd)</code>	dev engine jen s tastytrade (#623): IBKR vypnuto, žádné výpočty ani zápisy — stream chainu do cache s minutovým heartbeatem v logu engine	povolen — tasty snese souběžné streamy (ADR-0027), produkce nepřijde o minutu; news-engine se nespustí (mluví s TWS)
<code>scripts/seed-dev.ps1</code>	obnoví dev PG z nejnovější zálohy + zrcadlí <code>data/</code> → <code>data-dev/</code>	povolen

Pravidla:

- **Produkce použít výhradně** `main` . `start-prod.ps1 -Build` odmítne stavět z jiné větve nebo ze špinavého stromu (`-Force` = vědomé obejítí). Bez `-Build` se jen startují dřív postavené image. Dev použít libovolnou rozpracovanou větev.
- **Nasazení po mergi:** `git checkout main && git pull` , pak `.\scripts\start-prod.ps1 -Build` . Před nasazením, které sahá na schéma DB, vždy `.\scripts\backup-postgres.ps1` — izolace dev to nenahrazuje, je to druhá vrstva.
- Dev frontend nese v sidebaru oranžový badge **DEV** (build arg `VITE_GEXLENS_ENV`), ať se okna prohlížeče nespletou.
- Dev stack je jednorázový: rozbitý dev = `docker compose -f compose.dev.yml down -v` , smazat `data-dev/` , `seed-dev.ps1` ZNOVU.
- Dev engine má výchozí `clientId 2` (`GEXLENS_DEV_IBKR_CLIENT_ID`), aby se v TWS nepotkal s produkční jedničkou.
- `-LiveTasty` si bere konzervativní strop 2 000 subskripcí (`GEXLENS_TASTY_MAX_SUBSCRIPTIONS` ; měřeno 6 008 bez degradace, ADR-0027) — kapacita je pravděpodobně per účet, dev nesmí ujídat produkci. Vyžaduje dev tasty grant v `.env.dev` (#696). Pozor: cross-feed logika (shadow #613, fallback #614) se v tomto režimu ověřit nedá — potřebuje oba feedy vedle sebe.

10. Testy a CI

- **Python** (~160): jednotkové + golden (GEX, levels, heatmap módy, walls, CumΔ, profil), mock-based integrační (scheduler, hot zóna, runtime), PG integrační (v CI přes service kontejner), **e2e smoke** — deterministický referenční den přes celou pipeline engine→storage→API proti golden hodnotám.
- **Frontend** (~58): jednotkové (geometrie, barvy, contours, slice), komponentové (jsdom + testing-library, PointerEvent polyfill), Arrow round-trip loaderu, **e2e render smoke** (App

nad /replay balíkem), vizuální regresní snapshoty renderu.

- **CI** (GitHub Actions, na každý PR): python job (ruff, mypy strict, pytest + PostgreSQL service), frontend job (eslint, prettier, vitest, build). Výkonnostní testy s tvrdým limitem běží jen lokálně (`ci` env skip).

Bezpečnostní kontroly v CI

Job `security` na každém PR: gitleaks (celá historie — pozor, test s realisticky vypadajícím tajemstvím spadne i po přepsání souboru, dokud je v historii větve), pip-audit, npm audit.

Lokálně `pwsh scripts/security-scan.ps1`.

11. Známé limity účtu a otevřené body

Z [ADR-0001](#) (měřeno živě na účtu):

Limit	Hodnota	Dopad
Tick-by-tick streamy	5	Hot zóna degraduje z $ATM \pm 15$ na $\sim ATM \pm 1$; zbytek klasifikuje midpoint test. Rozšíření = IBKR Quote Booster.
Market data lines	100	Naměřený strop účtu (sonda #609). Původní údaj „ ≥ 150 “ v ADR-0001 neplatí — dávka 80 jede blízko stropu ($\sim 95-100/100$), <code>batch_size</code> proto nezvyšovat . Strukturální řešení přinese #616.
FOP OI	tick 588 nedodává nikdy; tick 101 funguje	VYŘEŠENO (issue #65, ADR-0001 v3): <code>IbOIFetcher</code> používá generic tick 101 pro OPT i FOP a čte hodnotu podle strany kontraktu (opačná strana = validní 0.0). Retry à 30 min + volume fallback zůstávají jako pojistka.

[ADR-0002](#): obálka strikes je grow-only (křídla se neztrácejí), strop šířky s alertem. [ADR-0003](#): multi-instrument orchestrace řízená watchlistem.

Sekundární datový zdroj — tastytrade/dxFeed (M7)

Naměřené limity a pasti feedu: **ADR-0027** (6 000+ symbolů na subskripci, REST ≥ 6 req/s, povinné KEEPALIVE, dekádová kolize futures candle symbolů). Přístup výhradně **OAuth2 scope** `read` — nikdy `/sessions`, nikdy `trade` (ADR-0025); granty oddělené pro dev a produkci. Shadow

mód (#613) porovnává oba feedy do `feed_comparison`, nic nepublikuje; vyhodnocení `scripts/feed_comparison_report.py`.

Konfigurace je jen přes `.env`, v Settings tastytrade není. Od fáze 2 (#614) se ale aktivní fallback **ukazuje v hlavičce** jantarovým chipem — tiché přepnutí zdroje zakazuje ADR-0025 pravidlo 5.

Pozor na klíče, které engine nečte. Model `Settings` má `extra="ignore"`, takže neznámý klíč se **tiše zahodí**. `GEXLENS_TASTY_CLIENT_ID` z `.env.example` nepoužívá nikdo (refresh flow posílá jen `refresh_token`

- `client_secret`) a `GEXLENS_DEV_TASTY_*` čte napřímo z prostředí jen sonda `scripts/tasty_probe.py`. Když nastavení „nezabírá“, ověř nejdřív, že ten klíč engine vůbec zná — seznam je v kapitole 4.

 `GEXLENS_TASTY_SHADOW` **dnes NENÍ jen vypínač měření.** Hlídá celý blok, pod kterým visí i křížová kontrola (#517 A), oba fallbacky (#614 2a/2b), OI fill (#664) i publikace ceny bez pipeline (#756). **Vypnutím si tiše vypneš odolnost proti výpadku IBKR**, aniž by to cokoli oznámilo — nedělej to, dokud nedoběhne #763 (rozdělení flagu na trvalou a měřicí část).

Provozní stav tasty větve (reconnecty DXLink, handshake, počet sledovaných symbolů, zapsané řádky) končí **jen v logu kontejneru** a v tabulce `feed_comparison ; V /status` je z něj `chain_source` a `spot_source`. Samostatná obrazovka je zadaná v #706.

Křížová kontrola feedů (#517 fáze A)

Heartbeat testuje jen TCP spojení a stall detektory (ADR-0015) měří pasivně stáří dat — incident 26.–27. 7. (15 h zmrzlé ATM greeks při tekoucích cenách) nezachytila ani jedna vrstva. Chyběla **nezávislá reference**: mlčí data, nebo mlčí trh? Běžící shadow ji dodává zadarmo, takže detektor nedělá **žádný request na IBKR a nebere žádnou market data line** — jen čte, co `compare_minute` stejně počítá.

Rozhoduje se z rozpadu kontraktů podle čerstvosti obou stran:

Situace	Verdikt	Chování
IBKR mlčí $\wedge$ tasty data má	<code>ibkr_suspect</code>	alert — tasty vylučuje „tichý trh“; farmu od mrtvých subskripcí rozliší až sonda fáze B
oba mlčí	<code>quiet</code>	ticho — nikdo neobchoduje, není co hlásit
tasty mlčí $\wedge$ IBKR data má	<code>tasty_suspect</code>	jen stav a log, bez alertu — viz níže

Situace	Verdikt	Chování
oba dodávají	ok	—

Proč zrovna 70 % / 3 minuty. Sweep obchází kontrakty po dávkách `batch_size`, takže **každou třetí minutu** vyskočí podíl „IBKR mlčí“ na ~58 % a hned zase spadne — rotační artefakt, ne porucha. Okamžitý podíl je proto jako signál nepoužitelný a naivní práh by alertoval 24/7. Měření na 3 016 minutách čisté shadow historie (13.–16. 8. 2026) — nejdelší souvislá série minut nad prahem:

práh	nejdelší série	počet epizod
50 %	2 min	696
60 %	2 min	42
70 %	2 min	2
80 %	1 min	1

Série tří minut v řadě nenastala při žádném prahu ani jednou. Podmínka „M minut v řadě“ tedy rotaci odfiltruje úplně; `_MINUTES` snižené na 2 by naopak plašilo.

Proč `tasty_suspect` nealertuje. Přehrání téže historie detektorem dalo 41 epizod „tasty mlčí ∧ IBKR data má“ — všechny v hodinách **21–06 UTC**. Příčina je konstrukční: dxFeed je **event-driven** (v klidu neposílá nic, okno stárí vyprší), IBKR sweep **poll-driven** (vrací poslední kotaci pořád dokola). V noci a v denní pauze CME (16:00–17:00 CT) je to normální stav, ne porucha — v alert kanálu by z toho bylo 41 planých poplachů. Stav se proto jen poznamená do `/status` a jednou na začátku epizody do logu.

Kontrolní měření po téhle úpravě: **3 053 minut čisté historie → 0 alertů.**

Stav je v **Settings** → **Stav enginu** (řádek *Křížová kontrola feedů*) a v `/status` jako `feed_crosscheck`. Když shadow neběží, pole ve statusu **chybí** — UI to ukáže jako „neměří se“, což je jiný stav než `ok`.

Fallback na tastytrade (#614 fáze 2a a 2b)

Když IBKR přestane posílat data, přebírá je tastytrade. Typická příčina není porucha, ale **souběh s mobilem**: market data jsou per uživatel, takže přihlášení do IBKR aplikace v telefonu přetáhne feed k sobě (error 10197) a engine zůstane připojený — jen mu přestanou chodit ticky.

Co	Kdy nastoupí tasty	Kdy se vrátí IBKR
Spot podkladu (2a)	30 s bez ticku	60 s souvislých dat
Opční řetěz — kotace, greeks, OI (2b)	verdict <code>ibkr_suspect</code> , tedy 3 min	5 čistých minut v řadě

Řetěz se spouští **verdiktem křížové kontroly**, ne vlastním prahem — dědí tak kalibraci měřenou na 3 016 minutách místo nového odhadu. Návrat vyžaduje `state == "ok" AND streak == 0` : samotné `ok` nestačí, protože detektor ho vrátí i pro minutu nad prahem, která zatím nenaplnila sérii do alertu. Stavby `quiet` a `insufficient` sérii jen nulují — tichý trh není důkaz uzdravení.

Co fallback vědomě NEoddá: kumulativní denní objem. Tasty ho ve stejné sémantice nemá, a dosadit nulu by znamenalo skokový záporný přírůstek přes celý řetěz. Po dobu fallbacku řetězu proto **stojí CumΔ a net objem** a ve snímcích jsou `NULL` — díra, kterou je vidět, místo čísla, které lže. Heatmapa, GEX profil, zdi i flip běží dál.

Přepnutí se hlásí alertem a jantarovým chipem v hlavičce; `/status` nese `chain_source` a `spot_source ( ibkr | tasty )`. `spot_source` se agreguje pesimisticky — stačí jeden instrument na fallbacku a status hlásí `tasty` , protože výpadek market data je vlastnost účtu.

⚠ Zbývá slepé místo: **mrtvá tasty větev dnes selže tiše** (#764). Stav `tasty_suspect` detektor umí, ale alert je vypnutý natvrdo kvůli 41 planým epizodám v noci. Když tedy tasty umře za běhu, pozná se to až při výpadku IBKR.

Start bez běžícího TWS (#756)

Engine na IBKR čeká **nejvýš** `GEXLENS_STARTUP_CONNECT_WAIT_S` (default 60 s) a pak rozjede zbytek i bez něj. Do #756 tu bylo nekonečné čekání, za kterým zůstalo úplně všechno — schéma DB, pipelines i tastytrade větev — takže po startu Windows bez spuštěné TWS neběželo nic.

Co platí po vypršení stropu:

- pipeline se **nezakládá** (discovery i subskripce jsou IBKR volání), ale existující se **neruší** — `plan.stop` se řídí watchlistem, ne spojením,
- tasty se odebírá i pro instrumenty **bez** pipeline, jinak by fallback neměl z čeho stavět,
- cenu publikuje samostatná smyčka, která pro symbol s běžící pipeline umlkne,
- `/status` chodí i bez pipelines a `connection` nese **skutečný** stav — „engine běží" a „IBKR je připojen" jsou dva různé stavy.

Pipeline se založí sama, jakmile se spojení objeví. Restart enginu není potřeba.

12. Diagnostika a údržba

Situace	Postup
Engine offline	<code>docker compose logs engine</code> — hledej stav ConnectionManageru; ověř TWS (API zapnuté, port, Trusted IP). Warning 2110/2103 = výpadek TWS↔IB, vyřeší se sám.
Prázdné GEX/walls	Zkontroluj <code>oi_eod</code> pro dnešek: <code>docker compose exec postgres psql -U gexlens -c "select date, count(*) from oi_eod group by 1 order by 1 desc limit 5"</code> — pokud dnešek chybí, engine archiv opakuje à 30 min (CME publikuje OI ráno).
Ticker z watchlistu nesbírá	<code>`docker compose logs engine</code>
Vysoké <code>Repair</code> / <code>Stale</code>	Konkrétní kontrakty bez dat — často nelikvidní křídla; zvyš <code>BATCH_TIMEOUT_S</code> nebo zmenši obálku.
Disk roste	Retention běží nočně; ručně: smaž staré partice v <code>./data</code> (nikdy <code>oi_eod</code>).
Reset prostředí	<code>docker compose down</code> , smaž <code>./data</code> (přijdeš o 14denní okno, ne o OI archiv ve volume <code>pgdata</code>), <code>docker compose up -d --build</code> .
Málo dat po restartu	Writer navazuje na rozepsaný den — mezera zůstane jen za dobu výpadku.
Změna portu TWS	Settings v aplikaci, nebo <code>.env</code> + <code>docker compose up -d engine</code> .
Graf zamrzl, engine hlásí connected	Souběh s mobilem (error 10197) — market data jsou per uživatel. Od #614 přebírá data tastytrade: do 30 s cena, do 3 min celý řetěz; v hlavičce svítí jantarový chip. Zamrzne-li i tak, ověř tasty větev: <code>`docker compose logs engine</code>
CumΔ stojí, zbytek jede	Očekávané chování při fallbacku řetězu (#614) — tasty denní objem v sémantice IBKR nedodává, tak se nevymýšlí. Skončí návratem na IBKR.
V logu není ani řádek <code>tasty</code>	Chybí grant v <code>.env</code> (<code>GEXLENS_TASTY_CLIENT_SECRET</code> , <code>_REFRESH_TOKEN</code>) nebo je vypnutý <code>GEXLENS_TASTY_SHADOW</code> — ten dnes hlídá i fallbacky, viz kap. 11.
<code>Error 200</code> , <code>reqId</code> 5/6 po startu	Známé (#734): <code>BRFUPDN</code> a <code>DJ</code> nemají celofeedovou pásku. Není to porucha, zprávy tečou z <code>BRFG</code> a <code>DJNL</code> .

Situace	Postup
Backfill zpráv	<code>python scripts/alpaca_news_backfill.py --dry-run</code> (změří objem), pak <code>--from 2024-07-28</code> — historie headlines z Alpaca s filtrem relevance (indexové ETF + mega caps; bez něj je ~85 % zpráv small caps bez vazby na index).

13. Zprovoznění od nuly — IBKR účet, TWS/Gateway

Jednorázový onboarding pro nové prostředí (převzato z issue #1, kde vznikl a byl odškrtnán při prvním zprovoznění 16. 7. 2026). Bez těchto kroků se engine nepřipojí, nebo dostane jen delayed data, která záměrně odmítá (SPEC 3.1 — Greeks z delayed dat nejsou spolehlivé).

13.1 Market data subskripce (Client Portal)

1. <https://www.interactivebrokers.com> → **Log In** → **Portal** (IBKR login + IB Key).
2. **Settings** → **User Settings** → **Market Data Subscriptions** → Configure (ozubené kolo).
3. **North America** → **Futures** → **CME Real-Time (NPL2)** — pokrývá ES/NQ futures i futures opce (FOP). Levná subskripce (~1,55 USD/měs.) prokazatelně stačí (ověřeno živě, ADR-0001).
4. Zkontroluj status **Non-Professional** (jinak výrazně vyšší poplatky).
5. (Až pro SPY/SPX — sekundární scope): **OPRA (US Options Exchanges)**, pro SPX index navíc **Cboe Streaming Market Indexes**.
6. Vývoj proti **paper účtu**: Settings → Account Settings → Paper Trading Account → *Share real-time market data with paper account* — jinak paper účet subskripce nevidí.

13.2 TWS nebo IB Gateway (musí běžet lokálně)

Engine se připojuje socketem na lokální TWS/Gateway (z kontejneru přes `host.docker.internal`), ne přímo na servery IBKR.

Varianta A — stávající TWS (nejrychlejší): Edit → Global Configuration → API → Settings → ☒ *Enable ActiveX and Socket Clients*, port **7496** live / **7497** paper, Trusted IPs `127.0.0.1` (vypne potvrzovací dialog), *Read-Only API* nechat **zapnuté** — GEXLens jen čte, nic neobchoduje.

Varianta B — dedikovaný IB Gateway (doporučeno pro trvalý provoz):

1. Stable Windows 64-bit: <https://www.interactivebrokers.com/en/trading/ibgateway-stable.php>
2. Login obrazovka: režim **IB API**, Live/Paper, přihlášení s IB Key.

3. Configure → Settings → API → Settings: port přepsat z 4001/4002 na **7496/7497** (nebo nechat a upravit `GEXLENS_IBKR_PORT` v `.env`), Trusted IPs `127.0.0.1`, Read-Only API zapnuté.
4. Configure → Settings → Lock and Exit → **Auto restart**, čas mimo seanci (např. 23:00) — jinak se TWS/GW jednou denně sám odhlásí a engine ztratí spojení přes noc. Jednou týdně (neděle) je i tak nutné plné ruční přihlášení — omezení IBKR.

13.3 Konflikt jednoho přihlášení ⚠

IBKR povoluje jedno přihlášení na username: Gateway + TWS (či mobil s trading permission, druhé PC) se stejným loginem současně = Gateway spadne. Řešení: druhý user v Client Portal (Settings → Account Settings → Users & Access Rights) — pozor, market data subskripce se platí per user. Pro start stačí varianta A.

13.4 Software na PC

- **Docker Desktop** s WSL2 backendem — celý stack (PostgreSQL, api, engine, frontend) běží přes `docker compose up -d` (kap. 3). Bez Dockeru: Python 3.12, Node.js ≥20, PostgreSQL 16 + `make run` (kap. 9).
- Volné místo: ~1 GB pro 14denní datové okno; WSL2 limit paměti viz `C:\Users\
<user>\.wslconfig` (`[wsl2] memory=6GB` — pojistka proti nafouknutí vmmem).

13.5 Ověření a denní provoz

```
Test-NetConnection 127.0.0.1 -Port 7496 # TcpTestSucceeded: True = API poslouchá
```

Každý obchodní den: TWS/Gateway běží a je přihlášený **před startem engineu**; stavová lišta aplikace ukazuje `connected :7496` a **• Live** (ne Offline). Diagnostika problémů: kap. 12.

14. Bezpečnost a nasazení na server

Výchozí stav aplikace počítá s tím, že běží na jednom PC za NATem. Přesun na VPS s veřejnou IP (#539) tenhle předpoklad ruší, proto proběhla prověrka #542. Tahle kapitola je její provozní výstup.

Tajemství

Dvě hodnoty musí být v lokálním `.env` (do repa nepatří, `.gitignore` je drží venku):

Proměnná	K čemu
GEXLENS_PG_PASSWORD	Heslo k PostgreSQL. Compose bez něj nenastartuje — žádný slabý default už neexistuje.
GEXLENS_API_TOKEN	Sdílené tajemství pro <code>/internal/*</code> (engine i news-engine → API) a <code>/backup/postgres</code> .

Vygeneruje je `pwsh scripts/init-secrets.ps1` — skript je idempotentní, existující hodnoty nepřepisuje a heslo rovnou přepíše i v běžícím PostgreSQL (`ALTER USER`). To je nutné: `POSTGRES_PASSWORD` se uplatní jen při prvním `initdb`, takže na existujícím volume by samotná změna v `.env` znamenala, že se služby k DB nepřipojí.

Token do UI patří jen kvůli stažení zálohy — vkládá se do pole **Settings** → **Záloha databáze** → **API token** a zůstává v localStorage prohlížeče. V image frontendu být nesmí, ten si stáhne kdokoli.

Síť

`compose.yml` nepublikuje nic na `0.0.0.0`. Adresu řídí `GEXLENS_BIND_ADDR` (default `127.0.0.1`); na serveru sem patří Tailscale IP. **Nikdy** `0.0.0.0` — Docker zapisuje publikaci do řetězce `DOCKER`, který se na Linuxu vyhodnocuje před UFW, takže port publikovaný bez adresy je veřejný i se „zapnutým firewallem“.

Prohlížeč mluví jen s nginx (`:8080`), který proxuje `/api` → `api:8000` včetně WebSocketu. Port API i PostgreSQL zůstávají publikované na loopback jen pro nástroje na hostiteli (zálohy, sondy) — pro provoz je potřeba nemá.

Checklist před nasazením na VPS

1. `pwsh scripts/init-secrets.ps1` na serveru; ověřit, že `.env` má práva 600.
2. `GEXLENS_BIND_ADDR` = Tailscale IP serveru.
3. `GEXLENS_ALLOWED_ORIGINS` = adresa UI, pod kterou se bude otevírat (jinak prohlížeč zablokuje fetch a WS handshake skončí na kontrole Origin).
4. UFW: povolit jen SSH a Tailscale; ověřit `nmap` z venku, že 8080/8000/55432 nejsou vidět (kontrolovat zvenčí, ne `ufw status` — viz past s DOCKER chainem).
5. SSH: klíče, `PasswordAuthentication no`, root login zakázaný.
6. IB Gateway: **VNC nikdy veřejně**, jen přes tunel.
7. Zálohy: `scripts/backup-postgres.ps1` (nebo cron s `pg_dump`) mimo server, šifrovaně — dump obsahuje celý OL archiv a historii setupů.

8. `docker compose up -d --build`, pak ověřit, že engine publikuje (`/status` ukazuje `engine: online`) — chybějící token se pozná tak, že UI zůstane bez živých dat a engine loguje chybu hned při startu.

Návrat zpět (rollback)

Rotace hesla nesahá na data — mění jen přihlašovací údaj, `pgdata` volume ani parquety se nedotkne. Vrátit ji lze kdykoli:

```
docker exec $(docker compose ps -q postgres) psql -U gexlens -d gexlens `
-c "ALTER USER gexlens PASSWORD 'gexlens';"
```

Funguje vždy, i když se heslo v `.env` rozejde s databází nebo `.env` ztratíš: `psql` uvnitř kontejneru chodí přes unix socket, který heslo nevyžaduje. Stejným příkazem se dá nastavit i nová hodnota, když je potřeba sladit `.env` s běžícím serverem. Po změně restartovat stack (`docker compose up -d`) — běžící kontejnery drží spojení se starým heslem.

Při revertu změn #542 na tohle nezapomenout. Starší `compose.yml` má `POSTGRES_PASSWORD: gexlens` natvrdo, takže po návratu ke starému kódu se služby k databázi s rotovaným heslem nepřipojí. Pořadí: nejdřív vrátit heslo příkazem výše, pak revertovat kód.

Opakovaná prověrka

`pwsh scripts/security-scan.ps1` spustí gitleaks (celá historie), pip-audit a npm audit; s `-Images` navíc trivy nad postavenými images. První tři běží i v CI jako job `security` na každý PR. Nálezy trivy jsou zpravidla OS balíky base image — řeší je rebuild s čerstvou bází, ne zásah do kódu, proto v CI nejsou.

Interní dokument. Uživatelská příručka: `UZIVATELSKY-MANUAL.md` (dostupná i v aplikaci jako Wiki).